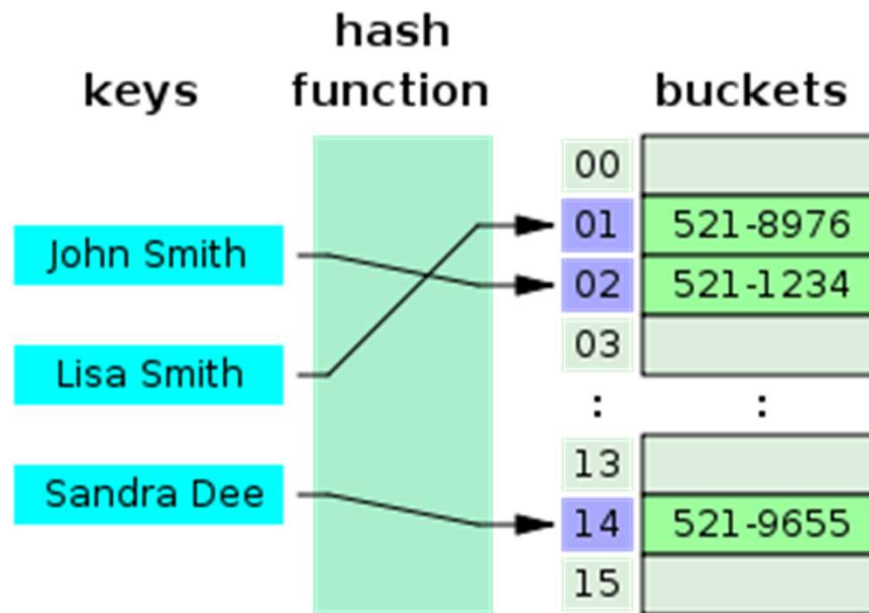


Hashing



So Far

Structure	add	find	remove	notes
unsorted array	$O(1)$	$O(n)$	$O(n)$	remove is expensive only because you have to first find the value
sorted array	$O(n)$	$O(\log n)$	$O(n)$	add and remove are expensive because you have to shift values
unsorted linked list	$O(1)$	$O(n)$	$O(n)$	remove is expensive only because you have to first find the value
sorted linked list	$O(n)$	$O(n)$	$O(n)$	add and remove are expensive because you have to find the right spot
binary search tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	assuming tree is balanced

- Obviously the well-behaved structure here is the binary search tree because everything becomes $O(\log n)$.
- That's why the Java class libraries include tree implementations of sets and maps.
- But it turns out you can do even better. With a structure known as a hash table, you can get $O(1)$ for each of these operations.

Objectives

- Basic Concept
- Hash Functions
- Collision Resolution
- Runtimes

Why Hashing?

The preceding chapters introduced search trees. An element can be found in $O(\log n)$ time in a well-balanced search tree.

Is there a more efficient way to search for an element in a container? This chapter introduces a technique called **hashing**. You can use hashing to implement a map or a set to search, insert, and delete an element in $O(1)$ time.

Introduction

- Suppose there is a set of data **M**
- There is a set of data that we actually care about storing **D**, where **D** << **M**
- What is our ideal data structure?
- The operation should run in $O(1)$ time
- What are some difficulties with this?

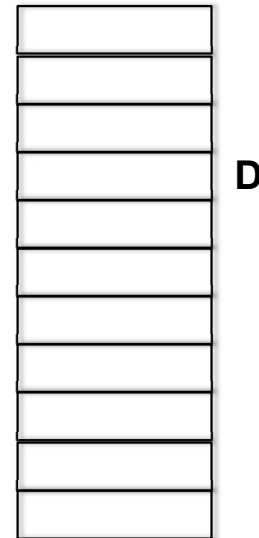
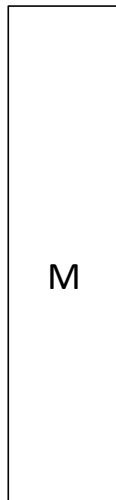
Hashing

- **Memory: The Hash Table**

- Consider an array of size $c * D$
- Each index in the array corresponds to *some element* in **M** that we want to store.
- The data in **D** does not need any particular ordering

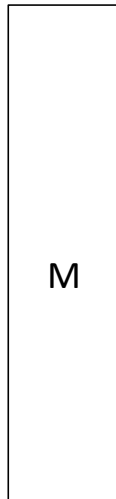
THE HASH TABLE

- **How can we do this?**



THE HASH TABLE

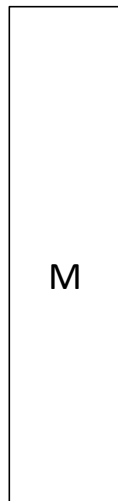
- **What is the problem here?**



Apple	D
Pear	
Orange	
Durian	
Kumquat	

THE HASH TABLE

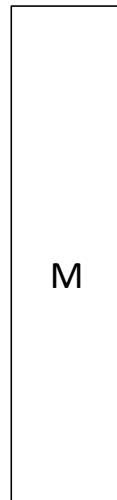
- **What is the problem here?**
 - Takes $O(D)$ time to find the word in the list!



Apple	D
Pear	
Orange	
Durian	
Kumquat	

THE HASH TABLE

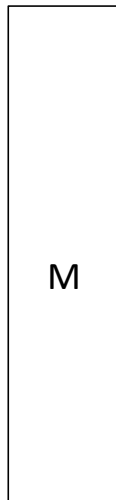
- **What is the problem here?**
 - Takes $O(D)$ time to find the word in the list
 - Same problem with sorted arrays!



Apple	D
Durian	
Kumquat	
Orange	
Pear	

THE HASH TABLE

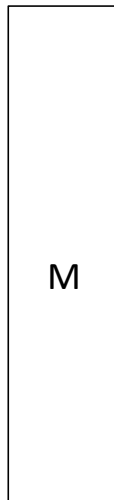
- **What is another solution?**



Apple	D
Pear	
Orange	
Durian	
Kumquat	

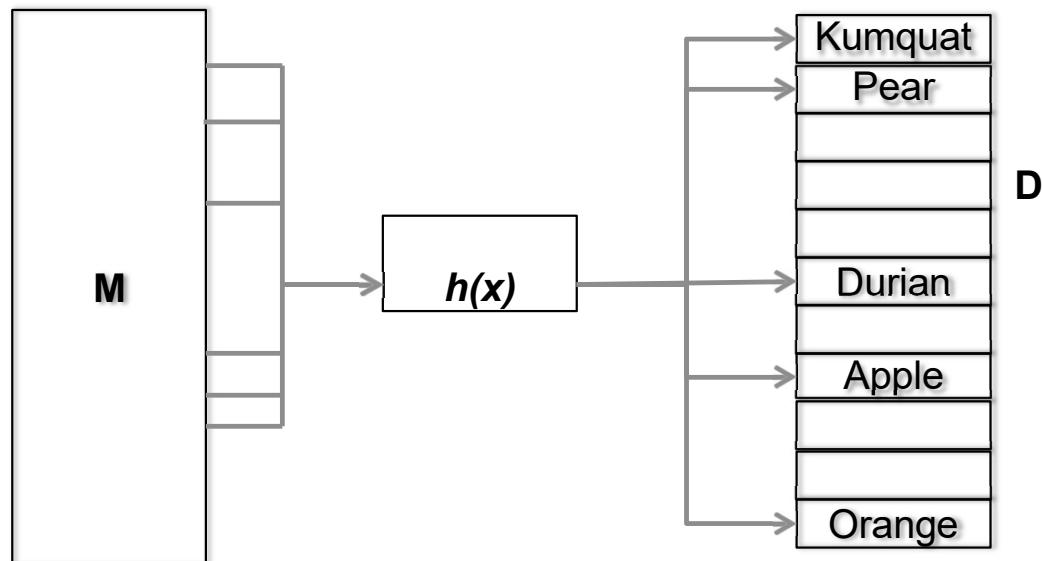
THE HASH TABLE

- **What is another solution?**
 - Random mapping



THE HASH TABLE

- What about a pseudo-random mapping?



Map

A *map* is a data structure that stores entries. Each entry contains two parts: *key* and *value*. The key is also called a *search key*, which is used to search for the corresponding value. For example, a dictionary can be stored in a map, where the words are the keys and the definitions of the words are the values.

A map is also called a *dictionary*, a *hash table*, or an associative array. The new trend is to use the term map.

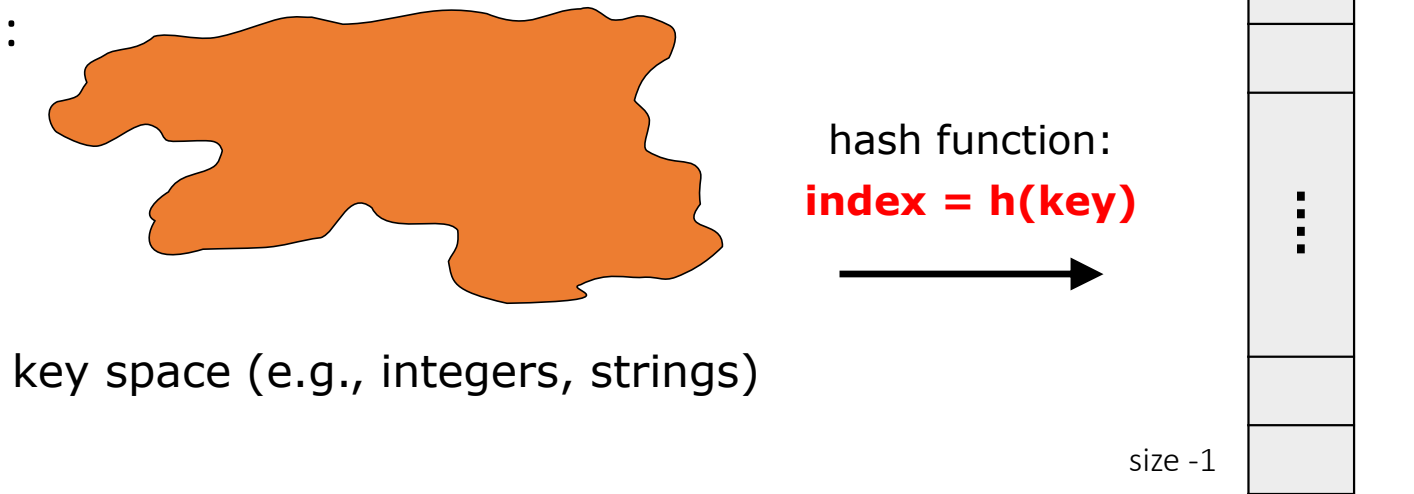


Map key to an index in HashTable

Hash Tables

A hash table is an array of some fixed size

Basic idea:



The goal:

What Is Hashing?

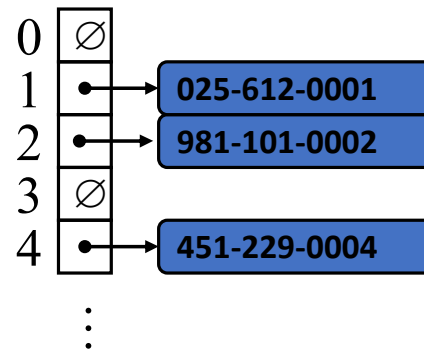
If you know the index of an element in the array, you can retrieve the element using the index in $O(1)$ time. So, can we store the values in an array and use the key as the index to find the value? The answer is yes if you can map a key to an index.

The array that stores the values is called a **hash table**. The function that maps a key to an index in the hash table is called a **hash function**.

Hashing is a technique that retrieves the value using the index obtained from key without performing a search.

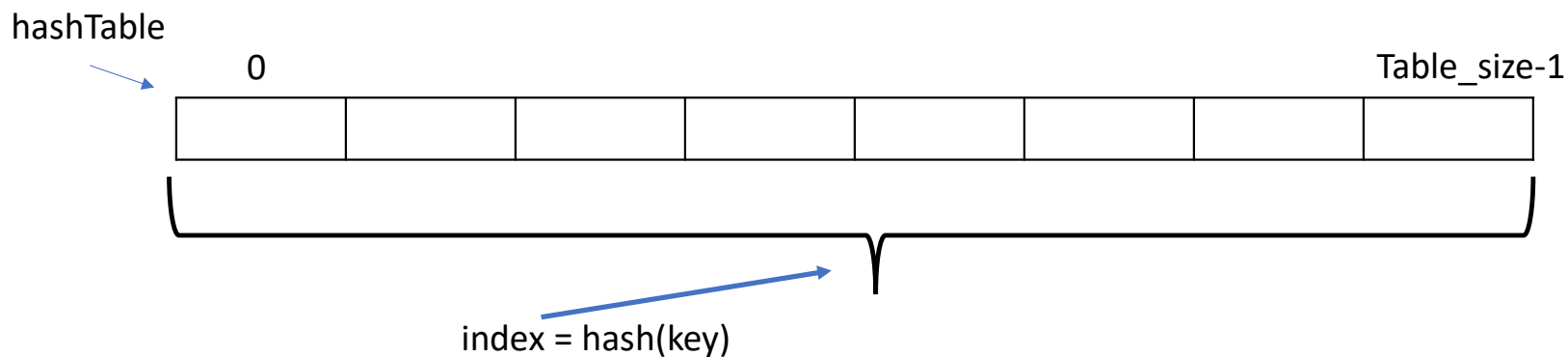
More General Kinds of Keys

- But what should we do if our keys are not integers in the range from 0 to $N - 1$?
 - Use a **hash function** to map general keys to corresponding indices in a table.
 - For instance, the last four digits of a Social Security number.



Hashing

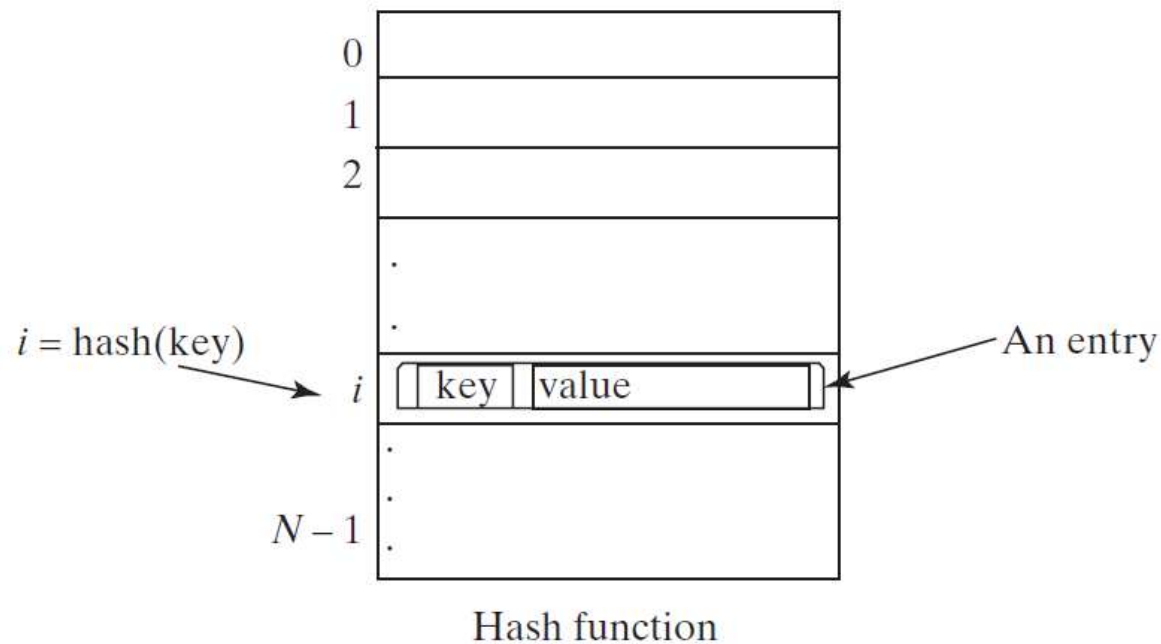
- A technique that determines an index into a table using only an entry's search key
- Hash function
 - Takes a search key and produces the integer index of an element in the hash table
 - Search key is mapped, or hashed, to the index



Hash Function and Hash Codes

A typical hash function:

1. first converts a search key to an integer value called a *hash code*,
2. Compresses the hash code into an index to the hash table.



Ideal Hashing

- Simple algorithms for the dictionary operations that add and retrieve

□ `put(k, v):`

insert entry (k, v) into the map M; if key k is not already in M,
then return null; else, return old value associated with k

□ `get(k):` if the map M has an entry with key k, return its associated value; else, return null

Hash Functions and Hash Tables



- A **hash function** h maps keys of a given type to integers in a fixed interval $[0, N - 1]$
- Example:
$$h(x) = x \bmod N$$

is a hash function for integer keys
- The integer $h(x)$ is called the **hash value** of key x
- A **hash table** for a given key type consists of
 - Hash function h
 - Array (called table) of size N
- When implementing a map with a hash table, the goal is to store item (k, o) at index $i = h(k)$

Typical Hashing

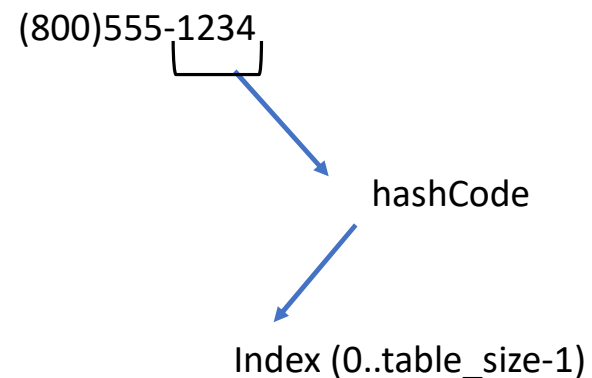
- Typical hash functions perform two steps:
 - Convert search key to an integer
 - Called the hash code.
 - Compress hash code into the range of indices for hash table.

Algorithm `getHashIndex(phoneNumber)`

// Returns an index to an array of tableSize elements.

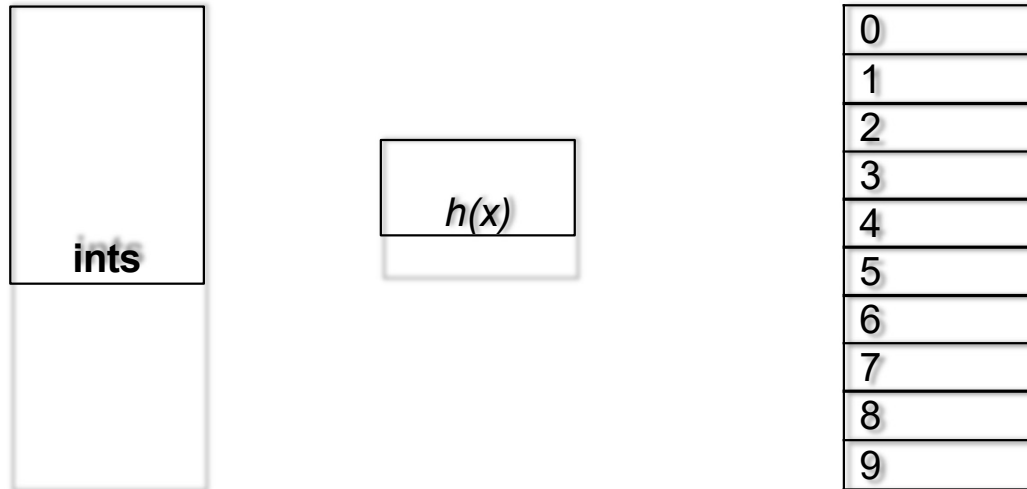
i = last four digits of phoneNumber

return $i \% \text{tableSize}$



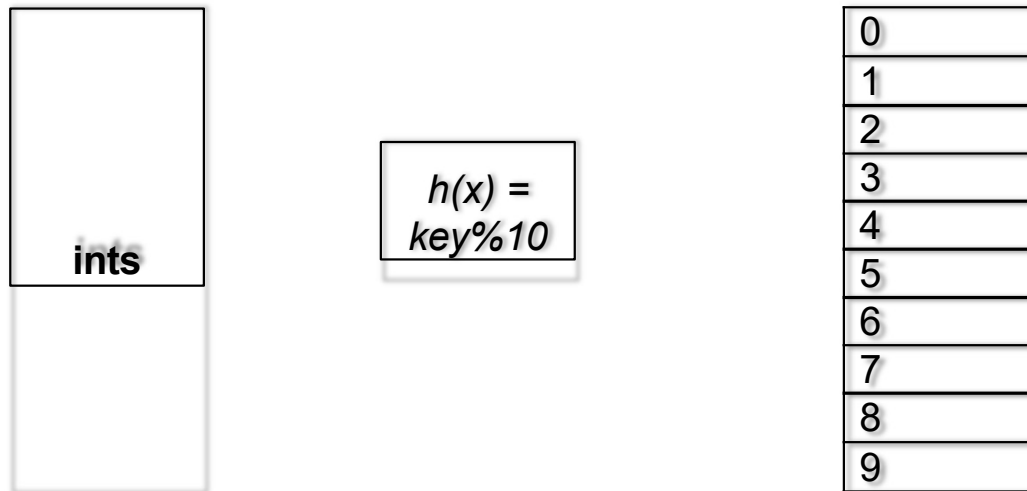
HASH EXAMPLE

- **Let's consider an example. We want to save 10 numbers from all possible Java ints**
 - What is a simple hash function?



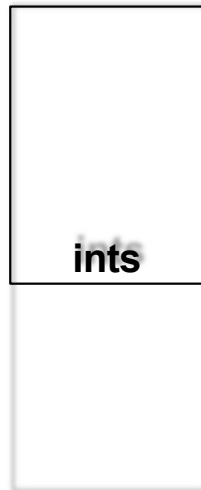
HASH EXAMPLE

- **Let's consider an example. We want to save 10 numbers from all possible Java ints**
 - What is a simple hash function?



HASH EXAMPLE

- **Let's insert(519) table**

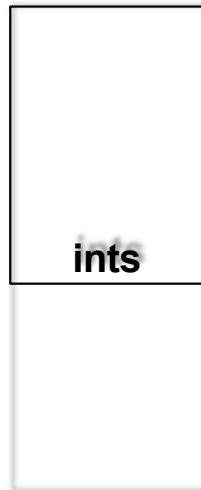


$$h(x) = \text{key} \% 10$$

0
1
2
3
4
5
6
7
8
9

HASH EXAMPLE

- **Let's insert(519) table**
 - Where does it go?

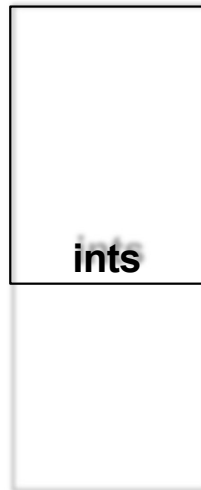


$$h(x) = \text{key} \% 10$$

0
1
2
3
4
5
6
7
8
9

HASH EXAMPLE

- **Let's insert(519) table**
 - Where does it go?
 - $519 \% 10 =$

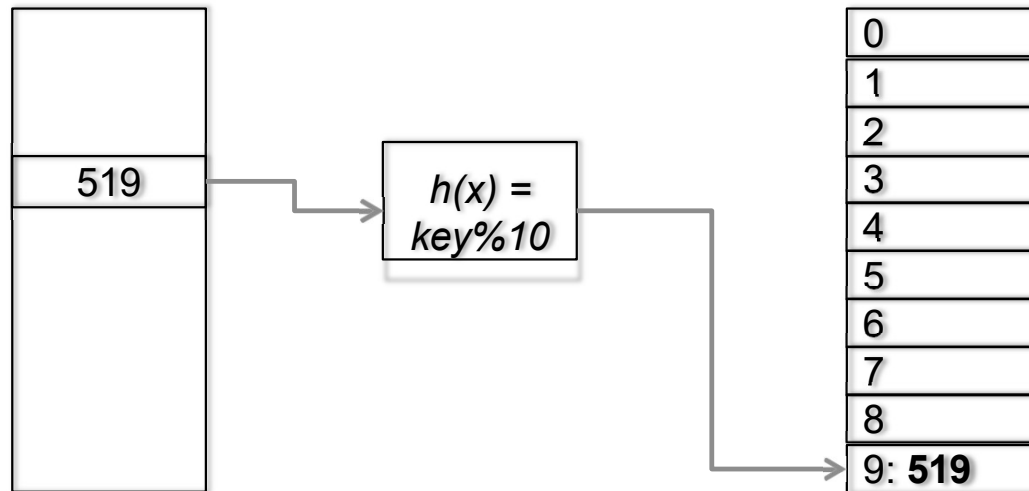


$$h(x) = \text{key} \% 10$$

0
1
2
3
4
5
6
7
8
9

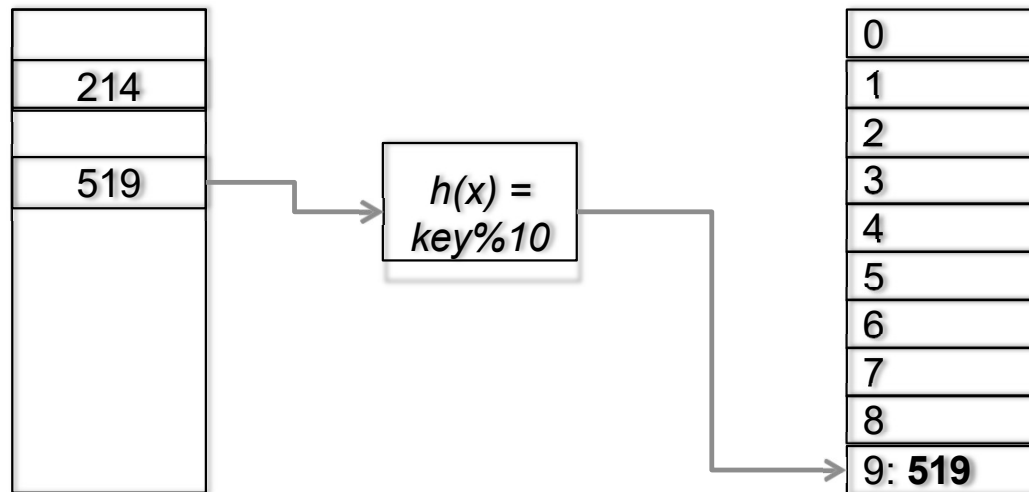
HASH EXAMPLE

- **Let's insert(519) table**
 - Where does it go?
 - $519 \% 10 = 9$



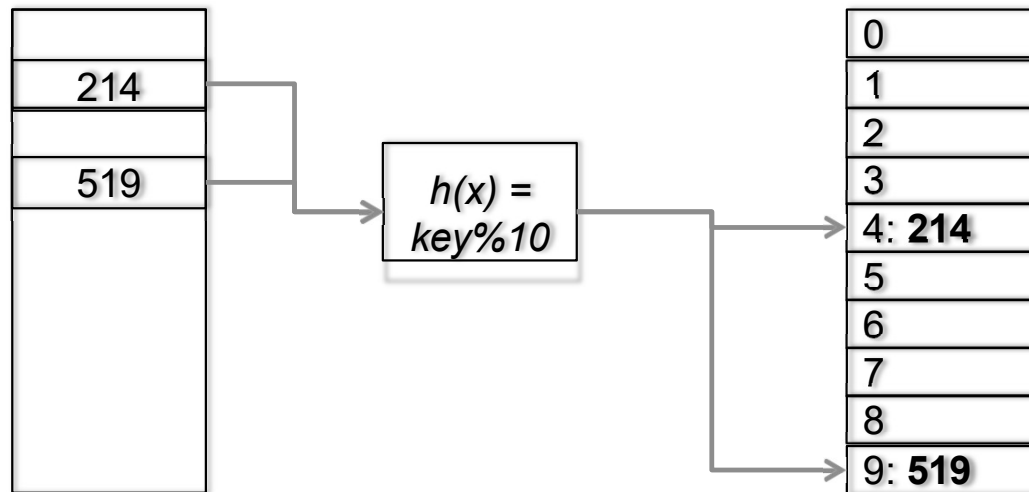
HASH EXAMPLE

- **Insert(214)**



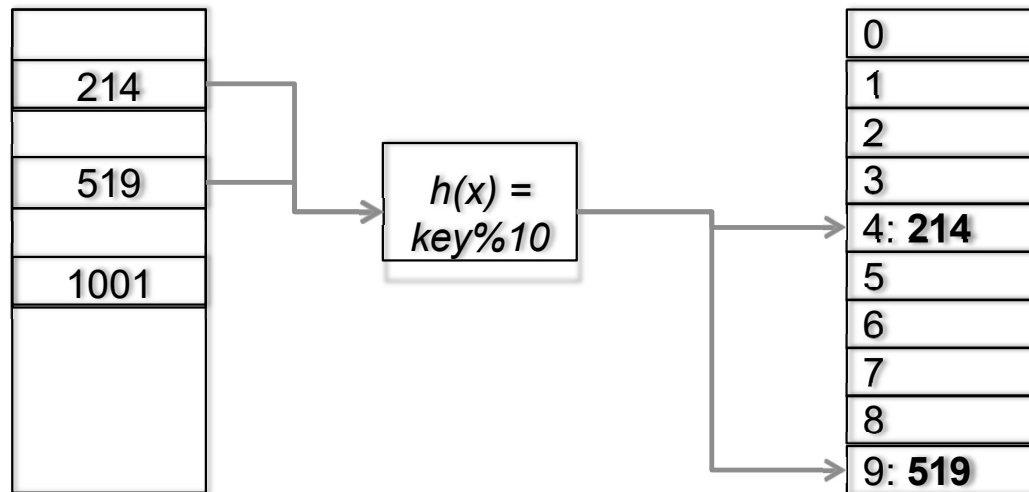
HASH EXAMPLE

- **Insert(214)**



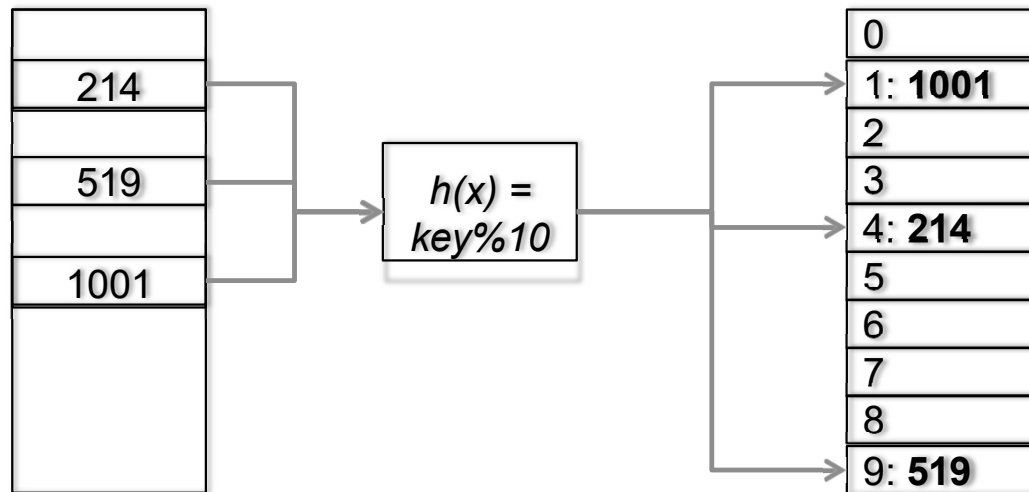
HASH EXAMPLE

- **insert(1001)**



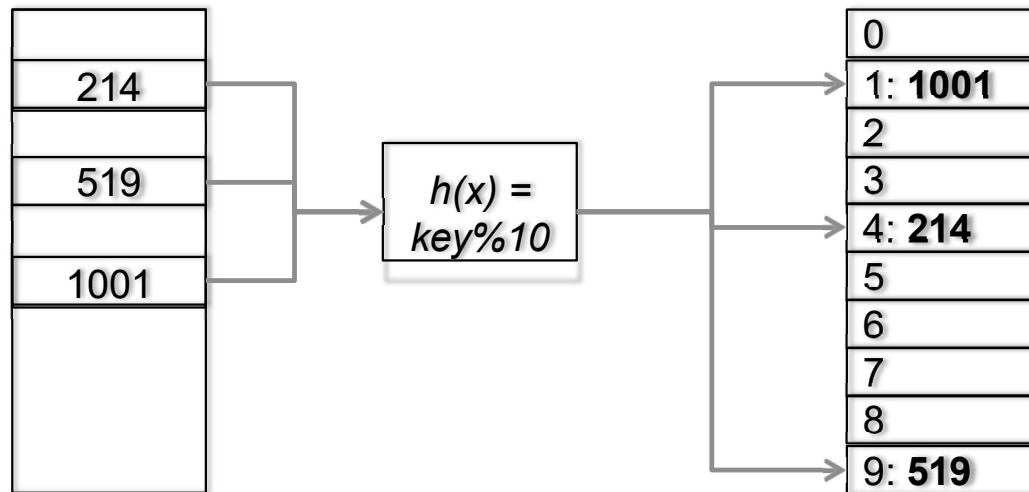
HASH EXAMPLE

- **insert(1001)**



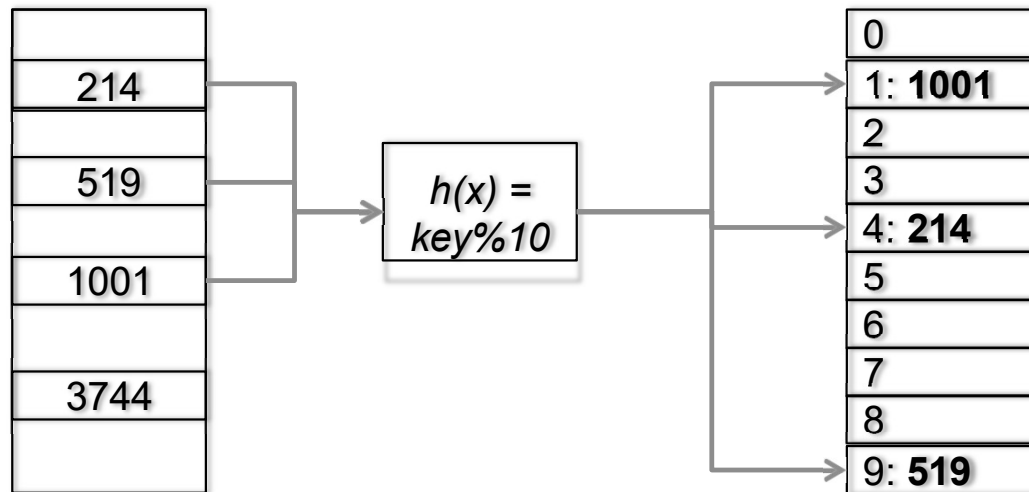
HASH EXAMPLE

- Is there a problem here?



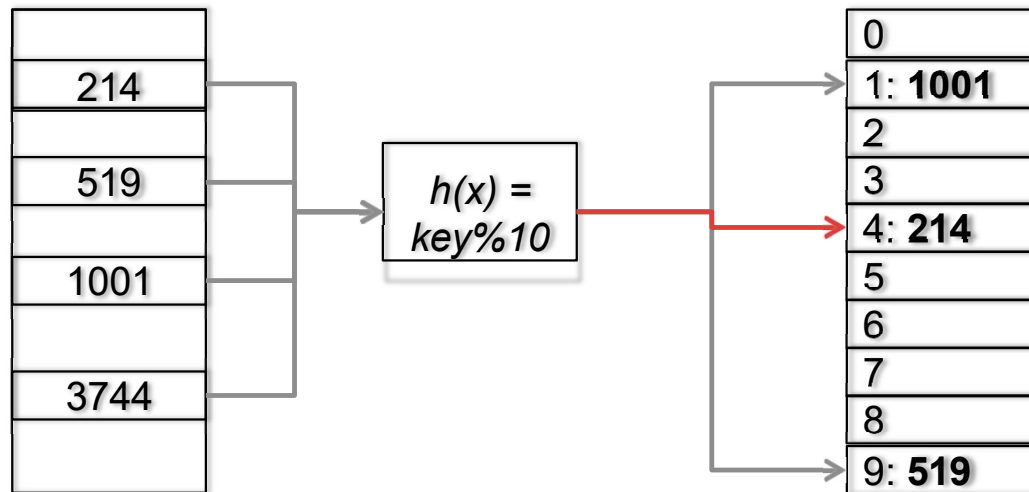
HASH EXAMPLE

- Is there a problem here?
 - insert(3744)



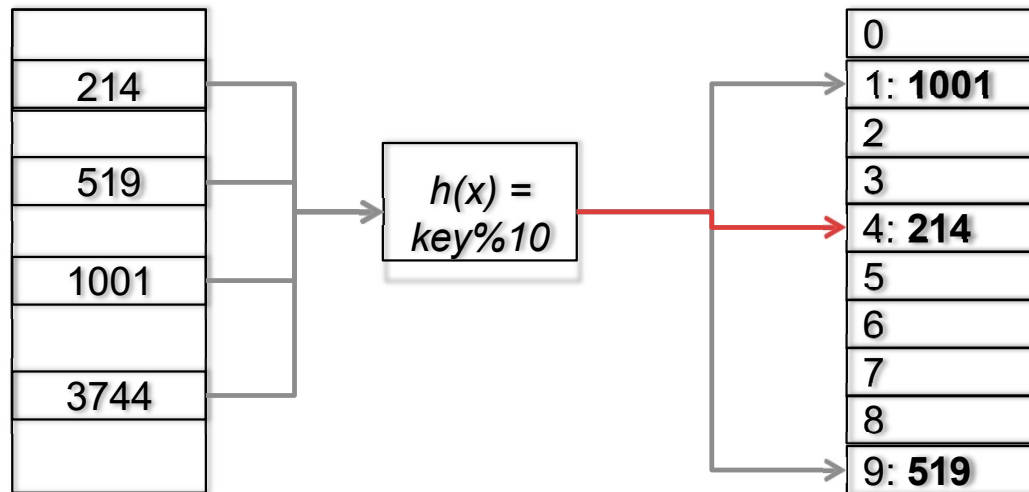
HASH EXAMPLE

- Is there a problem here?
 - insert(3744)



HASH EXAMPLE

- **Is there a problem here?**
 - insert(3744)
 - This is called a collision!



Hashing

- A perfect hash function
 - Maps each search key into a unique location of the hash table
 - Possible if all the search keys are known
- Collisions
 - Occur when the hash function maps more than one item into the same array location
- Collision-resolution schemes
 - Assign locations in the hash table to items with different search keys when the items are involved in a collision
- Requirements for a hash function
 - Be easy and fast to compute
 - Place items evenly throughout the hash table

Computing Hash Codes

- Java's base class **Object** has a method **hashCode** that returns an integer hash code
 - By default returns memory address of the object
- A class should define its own version of **hashCode**
 - Override: whenever the equals() method is overridden to ensure two equal objects return the same hashcode
 - Multiple execution of a program with same key should give same integer
 - Two unequal objects may have the same hashcode, must avoid too many such cases

A hash code for a string

➤ Approach 1:

- Using a character's Unicode integer is common

➤ Approach 2:

- Generate a hash code that takes the position of characters into consideration.
- Let the hash code be:

$$s_0 * b^{(n-1)} + s_1 * b^{(n-2)} + \dots + s_{n-1}, \text{ where } s_i \text{ is } s.\text{charAt}(i)$$

- The expression is a polynomial for some positive value (b).

■ Polynomial hash code

$$(\dots((s_0 * b + s_1) * b + s_2) * b + \dots + s_{n-2}) * b + s_{n-1}$$

Hash Code for a Primitive type

- If data type is **int (32 bits)** → `Integer.hashCode()`
 - Use the key itself
- For **byte (8 bits)**, **short (16 bits)**, **char (16 bits)**:
 - Cast as **int**
- **For long (64 bits)**
 - Casting from long to int (not good idea)
 - All keys that differ in only the first 32 bits will have the same hash code.
 - To take the first 32 bits into consideration (folding)
 - divide the 64 bits into two halves (bit-wise right-shift operator)
 - perform the exclusive-or operation to combine the two halves
 - Hash code for long key:
`int hashCode = (int)(key ^ (key >> 32));`

➤ Other primitive types (Manipulate internal binary representations)

- Hash code for float key (32 bits) → `Float.hashCode()`:

`Float.floatToIntBits(key)` → `Float.floatToIntBits(12.2f)` → 1094923059

`Float.floatToIntBits(2.2f)` → 1074580685

- Hash code for double key (64 bits) → `Double.hashCode()`:

`long bits = Double.doubleToLongBits(key);`

`int hashCode = (int)(bits ^ (bits >> 32));`

What to Hash?

We will focus on two most common things to hash: **ints** and **strings**

If you have objects with several fields, it is usually best to hash most of the "identifying fields" to avoid collisions:

```
class Person {  
    String firstName, middleName, lastName;  
    Date birthDate;  
    ...  
}
```



use these four values

An inherent trade-off:

hashing-time vs. collision-avoidance

Compressing a Hash Code

- Common way to scale an integer
 - Use Java mod operator %: `hashCode % n`
- Best to use a prime number for n
- Prime numbers often give good distribution of hash values
- Java API implementation for `java.util.Map`
 - n is set to an integer power of 2
 - Use logical bit-wise and (&) compress the hashCode
Index = `hashCode & (n-1)` → index will be in range of 0..(n-1)

`n=4 , hashCode=11`

Resolving Collisions

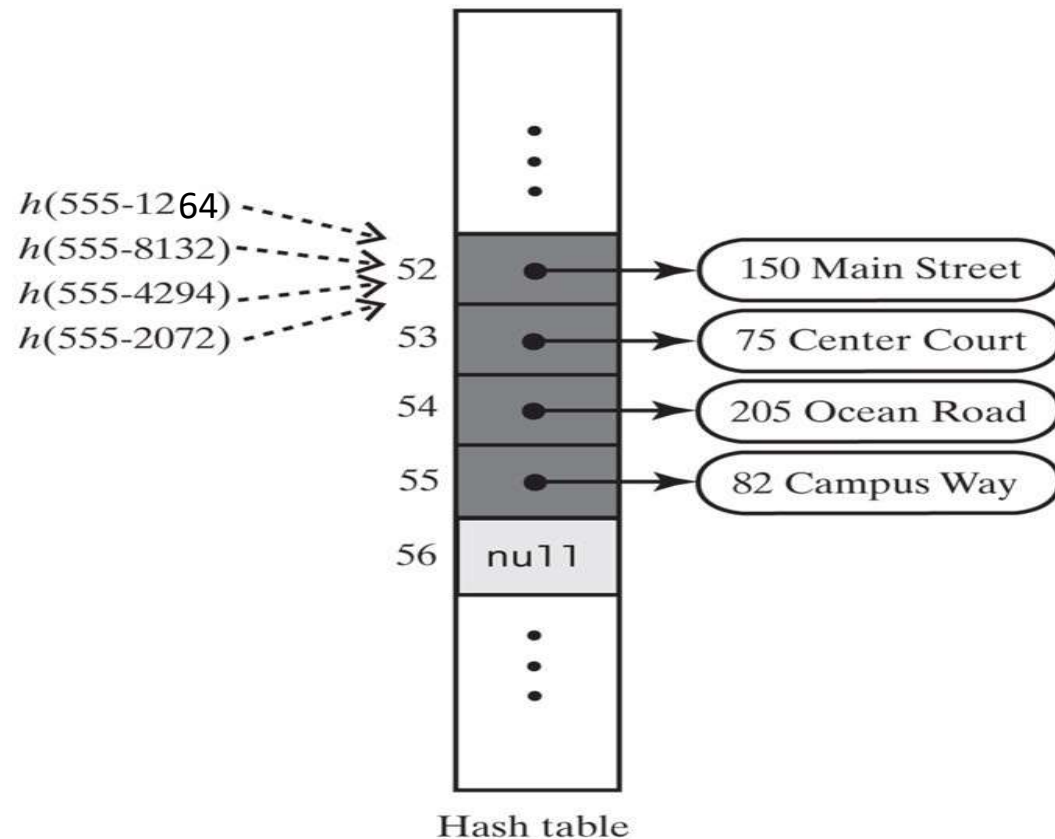
- Collision:
 - Hash function maps search key into a location in hash table already in use
- **How to rectify collisions?**
 - Think of a strategy for a few minutes
- **Possible solutions:**
 - Store in the next available space
 - Store both in the same space
 - Try a different hash
 - Resize the array

Resolving Collisions

- **Linear probing**

- Resolves a collision during hashing by examining consecutive locations in hash table
 - Beginning at original hash index
 - Find the next available one
- Table locations checked make up probe sequence
- If probe sequence reaches end of table, go to beginning of table (circular hash table)
 - Possible problem
 - Primary clustering

Linear Probing



© 2019 Pearson Education, Inc.

- The effect of linear probing after adding four entries whose search keys hash to the same index

Open Addressing: Linear Probing

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

How to deal with collisions?

$(h(\text{key}) + i) \% \text{TableSize}$ for $i \geq 0$

If $h(\text{key})$ is already full,

try $(h(\text{key}) + 1) \% \text{TableSize}$. If full,

try $(h(\text{key}) + 2) \% \text{TableSize}$. If full,

try $(h(\text{key}) + 3) \% \text{TableSize}$. If full...

Example: insert 38, 19, 8, 79, 10

Open Addressing: Linear Probing

0	
1	
2	
3	
4	
5	
6	
7	
8	38
9	

How to deal with collisions?

$(h(\text{key}) + i) \% \text{TableSize}$ for $i \geq 0$

If $h(\text{key})$ is already full,

try $(h(\text{key}) + 1) \% \text{TableSize}$. If full,

try $(h(\text{key}) + 2) \% \text{TableSize}$. If full,

try $(h(\text{key}) + 3) \% \text{TableSize}$. If full...

Example: insert 38, 19, 8, 79, 10

Open Addressing: Linear Probing

0	
1	
2	
3	
4	
5	
6	
7	
8	38
9	19

How to deal with collisions?

$(h(\text{key}) + i) \% \text{TableSize}$ for $i \geq 0$

If $h(\text{key})$ is already full,

try $(h(\text{key}) + 1) \% \text{TableSize}$. If full,

try $(h(\text{key}) + 2) \% \text{TableSize}$. If full,

try $(h(\text{key}) + 3) \% \text{TableSize}$. If full...

Example: insert 38, 19, 8, 79, 10

Open Addressing: Linear Probing

0	8
1	
2	
3	
4	
5	
6	
7	
8	38
9	19

How to deal with collisions?

$(h(\text{key}) + i) \% \text{TableSize}$ for $i \geq 0$

If $h(\text{key})$ is already full,

try $(h(\text{key}) + 1) \% \text{TableSize}$. If full,

try $(h(\text{key}) + 2) \% \text{TableSize}$. If full,

try $(h(\text{key}) + 3) \% \text{TableSize}$. If full...

Example: insert 38, 19, 8, 79, 10

Open Addressing: Linear Probing

0	8
1	79
2	
3	
4	
5	
6	
7	
8	38
9	19

How to deal with collisions?

$(h(\text{key}) + i) \% \text{TableSize}$ for $i \geq 0$

If $h(\text{key})$ is already full,

try $(h(\text{key}) + 1) \% \text{TableSize}$. If full,

try $(h(\text{key}) + 2) \% \text{TableSize}$. If full,

try $(h(\text{key}) + 3) \% \text{TableSize}$. If full...

Example: insert 38, 19, 8, 79, 10

Open Addressing: Linear Probing

0	8
1	79
2	10
3	
4	
5	
6	
7	
8	38
9	19

How to deal with collisions?

$(h(\text{key}) + i) \% \text{TableSize}$ for $i \geq 0$

If $h(\text{key})$ is already full,

try $(h(\text{key}) + 1) \% \text{TableSize}$. If full,

try $(h(\text{key}) + 2) \% \text{TableSize}$. If full,

try $(h(\text{key}) + 3) \% \text{TableSize}$. If full...

Example: insert 38, 19, 8, 79, 10

Linear Probing : Practice

Insert 44, 4, 16, 28, 21,26 into a table of size 11

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	

Load factor

The **load factor**, λ , of a hash table is calculated as

$$\lambda = \frac{n}{TableSize}$$

where n is the number of items currently in the table

Load Factor?

0	8
1	79
2	10
3	
4	
5	
6	
7	
8	38
9	19

Can the load factor when using linear probing ever exceed 1.0?

Nope!!

$$\lambda = \frac{n}{TableSize} = \frac{5}{10} = 0.5$$

Open Addressing: Other Operations

insert finds an open table position using a probe function

What about **find**?

- Must use same probe function to "retrace the trail" for the data
- Unsuccessful search when reach empty position

- [Demo](#)

What about **delete**?

- Must use "lazy" deletion. Why?

10	×	/	23	/	/	16	×	26
----	---	---	----	---	---	----	---	----

- Marker indicates "data was here, keep on probing"

Open Addressing: Linear Probing

0	8
1	79
2	10
3	
4	
5	
6	
7	
8	38
9	19

Now insert 18

What are the problems with this?

Need to go to 8, and then cycle through all of the entries until we get to empty or deleted entry

Open Addressing: Linear Probing

0	8
1	79
2	10
3	18
4	
5	
6	
7	
8	38
9	19

18 is placed at index 3.

- How do we search for 10?
- Need to go to 8, and then cycle through all of the entries until we find the element or find a blank space
- What if we need to add something that ends in 8?
 - It also ends up in this problem area
 - This is called **primary clustering**

Open Addressing: Quadratic Probing

We can avoid primary clustering by changing the probe function from just i to $f(i)$
 $(h(\text{key}) + f(i)) \% \text{TableSize}$

For **quadratic probing**, $f(i) = i^2$: for $i \geq 0$

0th probe: $(h(\text{key}) + 0) \% \text{TableSize}$

1st probe: $(h(\text{key}) + 1) \% \text{TableSize}$

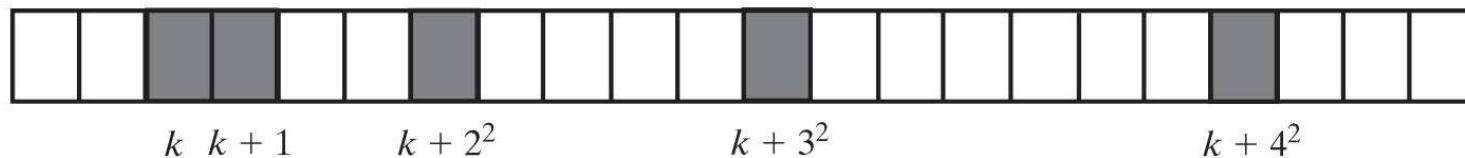
2nd probe: $(h(\text{key}) + 4) \% \text{TableSize}$

3rd probe: $(h(\text{key}) + 9) \% \text{TableSize}$

...

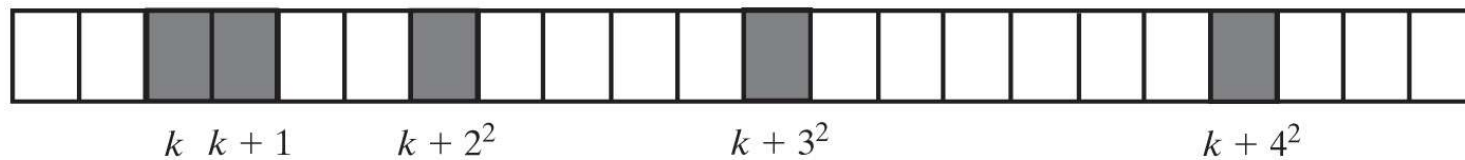
i^{th} probe: $(h(\text{key}) + i^2) \% \text{TableSize}$

Intuition: Probes quickly "leave the neighborhood"



Open Addressing with Quadratic Probing

- Linear probing looks at consecutive locations beginning at index k
- Quadratic probing:
 - Considers the locations at indices $k + j^2$
 - Uses the indices $k, k + 1, k + 4, k + 9, \dots$



© 2019 Pearson Education, Inc.

A probe sequence of length five using quadratic probing

Possible problem:

Secondary clustering: entries that collide with an occupied entry use the same probe sequence

Quadratic Probing Example

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

TableSize = 10
insert(89)

Quadratic Probing Example

TableSize = 10

insert(89)

insert(18)

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	89

Quadratic Probing Example

0	
1	
2	
3	
4	
5	
6	
7	
8	18
9	89

TableSize = 10

insert(89)

insert(18)

insert(49)

Quadratic Probing Example

0	49
1	
2	
3	
4	
5	
6	
7	
8	18
9	89

TableSize = 10

insert(89)

insert(18)

insert(49)

$49 \% 10 = 9$ collision!

$(49 + 1) \% 10 = 0$

insert(58)

Quadratic Probing Example

0	49
1	
2	58
3	
4	
5	
6	
7	
8	18
9	89

TableSize = 10

insert(89)

insert(18)

insert(49)

insert(58)

$58 \% 10 = 8$ collision!

$(58 + 1) \% 10 = 9$ collision!

$(58 + 4) \% 10 = 2$

insert(79)

Quadratic Probing Example

0	49
1	
2	58
3	79
4	
5	
6	
7	
8	18
9	89

TableSize = 10

insert(89)

insert(18)

insert(49)

insert(58)

insert(79)

$79 \% 10 = 9$ collision!

$(79 + 1) \% 10 = 0$ collision!

$(79 + 4) \% 10 = 3$

Another Quadratic Probing Example

TableSize = 7

0	
1	
2	
3	
4	
5	
6	

Insert:

76 ($76 \% 7 = 6$)

40 ($40 \% 7 = 5$)

48 ($48 \% 7 = 6$)

5 ($5 \% 7 = 5$)

55 ($55 \% 7 = 6$)

47 ($47 \% 7 = 5$)

Another Quadratic Probing Example

0	
1	
2	
3	
4	
5	
6	76

TableSize = 7

Insert:

76 ($76 \% 7 = 6$)

40 ($40 \% 7 = 5$)

48 ($48 \% 7 = 6$)

5 ($5 \% 7 = 5$)

55 ($55 \% 7 = 6$)

47 ($47 \% 7 = 5$)

Another Quadratic Probing Example

0	
1	
2	
3	
4	
5	40
6	76

TableSize = 7

Insert:

76 ($76 \% 7 = 6$)

40 ($40 \% 7 = 5$)

48 ($48 \% 7 = 6$)

5 ($5 \% 7 = 5$)

55 ($55 \% 7 = 6$)

47 ($47 \% 7 = 5$)

Another Quadratic Probing Example

0	48
1	
2	
3	
4	
5	40
6	76

TableSize = 7

Insert:

76 ($76 \% 7 = 6$)

40 ($40 \% 7 = 5$)

48 ($48 \% 7 = 6$)

5 ($5 \% 7 = 5$)

55 ($55 \% 7 = 6$)

47 ($47 \% 7 = 5$)

Another Quadratic Probing Example

0	48
1	
2	5
3	
4	
5	40
6	76

TableSize = 7

Insert:

76 $(76 \% 7 = 6)$

40 $(40 \% 7 = 5)$

48 $(48 \% 7 = 6)$

5 $(5 \% 7 = 5)$

55 $(55 \% 7 = 6)$

47 $(47 \% 7 = 5)$

Another Quadratic Probing Example

0	48
1	
2	5
3	55
4	
5	40
6	76

TableSize = 7

Insert:

76 ($76 \% 7 = 6$)

40 ($40 \% 7 = 5$)

48 ($48 \% 7 = 6$)

5 ($5 \% 7 = 5$)

55 ($55 \% 7 = 6$)

47 ($47 \% 7 = 5$)

Another Quadratic Probing Example

0	48
1	
2	5
3	55
4	
5	40
6	76

**Will we ever get
a 1 or 4?!?**

TableSize = 7

Insert:

76 ($76 \% 7 = 6$)

40 ($40 \% 7 = 5$)

48 ($48 \% 7 = 6$)

5 ($5 \% 7 = 5$)

55 ($55 \% 7 = 6$)

47 ($47 \% 7 = 5$)

$(47 + 1) \% 7 = 6$ collision!

$(47 + 4) \% 7 = 2$ collision!

$(47 + 9) \% 7 = 0$ collision!

$(47 + 16) \% 7 = 0$ collision!

$(47 + 25) \% 7 = 2$ collision!

Another Quadratic Probing Example

0	48
1	
2	5
3	55
4	
5	40
6	76

insert(47) will always fail here. Why?

For all n , $(5 + n^2) \% 7$ is 0, 2, 5, or 6

Proof uses induction and

$$(5 + n^2) \% 7 = (5 + (n - 7)^2) \% 7$$

In fact, for all c and k ,

$$(c + n^2) \% k = (c + (n - k)^2) \% k$$

Quadratic Probing : Practice

Insert 44, 4, 16, 28, 21, 26 into a table of size 11

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	

Clustering Reconsidered

Quadratic probing does not suffer from primary clustering as the quadratic nature quickly escapes the neighborhood

But it is no help if keys initially hash the same index

- Any 2 keys that hash to the same value will have the same series of moves after that
- Called *secondary clustering*

We can avoid secondary clustering with a probe function that depends on the key:
double hashing

Open Addressing: Double Hashing

Idea:

Given two good hash functions $h1$ and $h2$, it is very unlikely that for some key, $h1(key) == h2(key)$
Why not probe using $h2(key)$?

For **double hashing**, $f(i) = i \cdot h2(key)$:

0th probe: $(h1(key) + 0 \cdot h2(key)) \% TableSize$

1st probe: $(h1(key) + 1 \cdot h2(key)) \% TableSize$

2nd probe: $(h1(key) + 2 \cdot h2(key)) \% TableSize$

...

i^{th} probe: $(h1(key) + i \cdot h2(key)) \% TableSize$

Crucial Detail:

We must make sure that $h2(key)$ cannot be 0

Double Hashing

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

$T = 10$ (TableSize)

Hash Functions:

$$h1(key) = key \bmod T$$

$$h2(key) = 1 + ((key/T) \bmod (T-1))$$

Insert these values into the hash table in this order. Resolve any collisions with double hashing:

13

28

33

147

43

Double Hashing

0	
1	
2	
3	13
4	
5	
6	
7	
8	
9	

$T = 10$ (TableSize)

Hash Functions:

$$h1(key) = key \bmod T$$

$$h2(key) = 1 + ((key/T) \bmod (T-1))$$

Insert these values into the hash table in this order. Resolve any collisions with double hashing:

13

28

33

147

43

Double Hashing

0	
1	
2	
3	13
4	
5	
6	
7	
8	28
9	

$T = 10$ (TableSize)

Hash Functions:

$$h1(key) = key \bmod T$$

$$h2(key) = 1 + ((key/T) \bmod (T-1))$$

Insert these values into the hash table in this order. Resolve any collisions with double hashing:

13

28

33

147

43

Double Hashing

0	
1	
2	
3	13
4	
5	
6	
7	33
8	28
9	

$T = 10$ (TableSize)

Hash Functions:

$$h1(key) = key \bmod T$$

$$h2(key) = 1 + ((key/T) \bmod (T-1))$$

Insert these values into the hash table in this order. Resolve any collisions with double hashing:

13

28

$$33 \rightarrow g(33) = 1 + 3 \bmod 9 = 4$$

147

43

Double Hashing

0	
1	
2	
3	13
4	
5	
6	
7	33
8	28
9	147

$T = 10$ (TableSize)

Hash Functions:

$$h1(key) = key \bmod T$$

$$h2(key) = 1 + ((key/T) \bmod (T-1))$$

Insert these values into the hash table in this order. Resolve any collisions with double hashing:

13

28

33

$$147 \rightarrow g(147) = 1 + 14 \bmod 9 = 6$$

43

Double Hashing

0	
1	
2	
3	13
4	
5	
6	
7	33
8	28
9	147

$T = 10$ (TableSize)

Hash Functions:

$$h1(key) = key \bmod T$$

$$h2(key) = 1 + ((key/T) \bmod (T-1))$$

Insert these values into the hash table in this order. Resolve any collisions with double hashing:

13

28

33

$$147 \rightarrow g(147) = 1 + 14 \bmod 9 = 6$$

$$43 \rightarrow g(43) = 1 + 4 \bmod 9 = 5$$

We have a problem:

$$\begin{array}{lll} 3 + 0 = 3 & 3 + 5 = 8 & 3 + 10 = 13 \\ & 3 + 15 = 18 & 3 + 20 = 23 \end{array}$$

Double Hashing Analysis

Because each probe is "jumping" by $g(\text{key})$ each time, we should ideally "leave the neighborhood" and "go different places from the same initial collision"

But, as in quadratic probing, we could still have a problem where we are not "safe" due to an infinite loop despite room in table

This cannot happen in at least one case:

For primes p and q such that $2 < q < p$

$$h_1(\text{key}) = \text{key} \% p$$

$$h_2(\text{key}) = q - (\text{key} \% q)$$

Double Hashing

Double hashing uses a secondary hash function on the keys to determine the increments to avoid the clustering problem.

$k = h_1(\text{key}) = \text{key} \% \text{table-size}$ (table-size=11)

$h_2(\text{key}) = 7 - \text{key} \% 7;$

Insert 44, 4, 16, 28, 21, 26 into a table of size 11

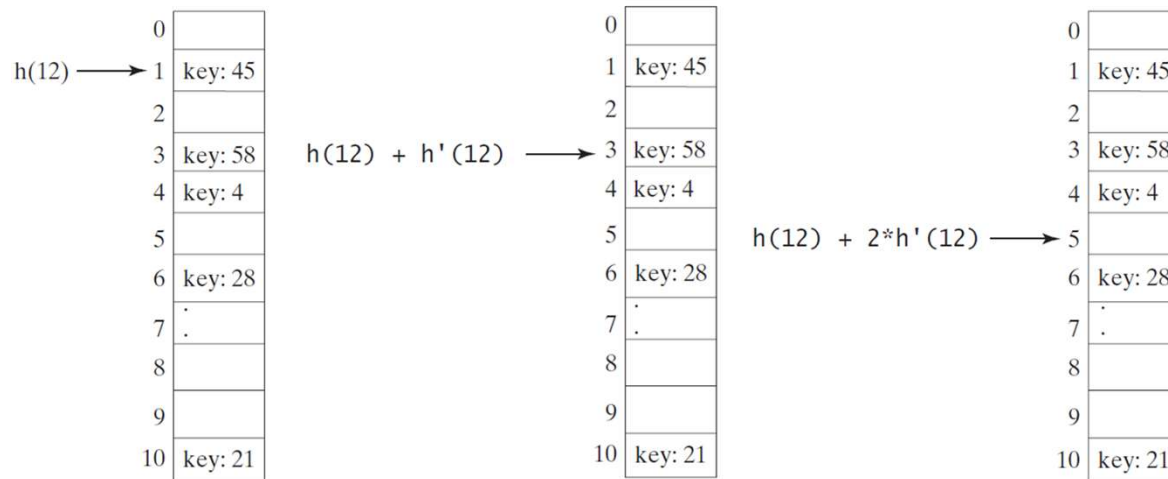
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	

Double Hashing

<https://liveexample.pearsoncmg.com/dsanimation/DoubleHashingeBook.html>

Double hashing uses a secondary hash function on the keys to determine the increments to avoid the clustering problem.

$$h'(k) = 7 - k \% 7;$$



Potential Problem with Open Addressing

- Recall each location is either occupied, empty, or available
 - Frequent additions and removals can result in no locations that are null
- Thus searching a probe sequence will not work
- Consider separate chaining as a solution

Resolving Collisions

- Approach 2: Restructuring the hash table
 - Changes the structure of the hash table so that it can accommodate more than one item in the same location
 - Buckets
 - Each location in the hash table is itself an array called a bucket
 - Separate chaining
 - Each hash table location is a linked list

Separate Chaining

0	/
1	/
2	/
3	/
4	/
5	/
6	/
7	/
8	/
9	/

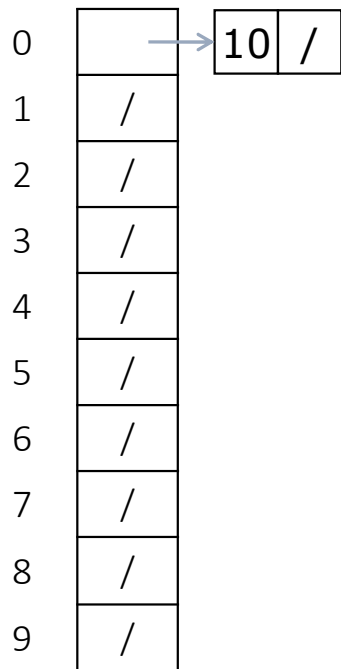
All keys that map to the same table location are kept in a linked list (a.k.a. a "chain" or "bucket")

As easy as it sounds

Example:

insert 10, 22, 86, 12, 42
with $h(x) = x \% 10$

Separate Chaining



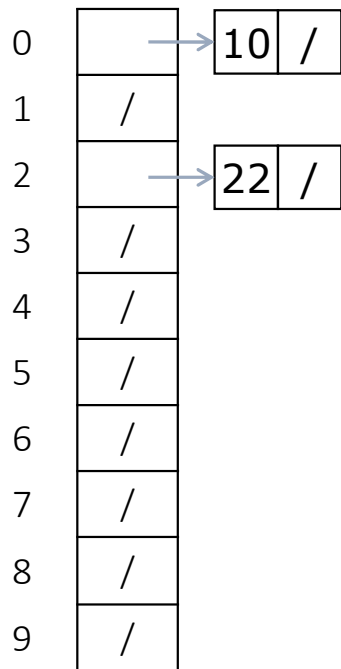
All keys that map to the same table location are kept in a linked list (a.k.a. a "chain" or "bucket")

As easy as it sounds

Example:

insert 10, 22, 86, 12, 42
with $h(x) = x \% 10$

Separate Chaining



All keys that map to the same table location are kept in a linked list (a.k.a. a "chain" or "bucket")

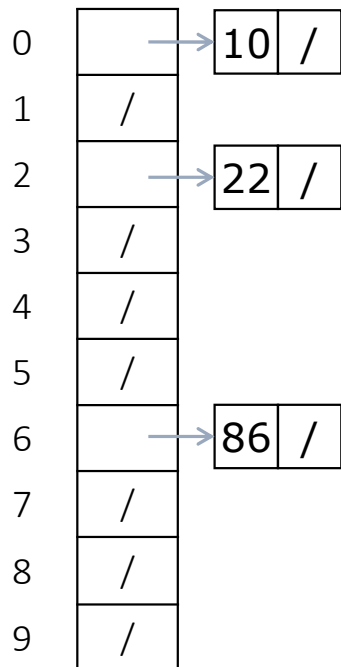
As easy as it sounds

Example:

insert 10, 22, 86, 12, 42

with $h(x) = x \% 10$

Separate Chaining



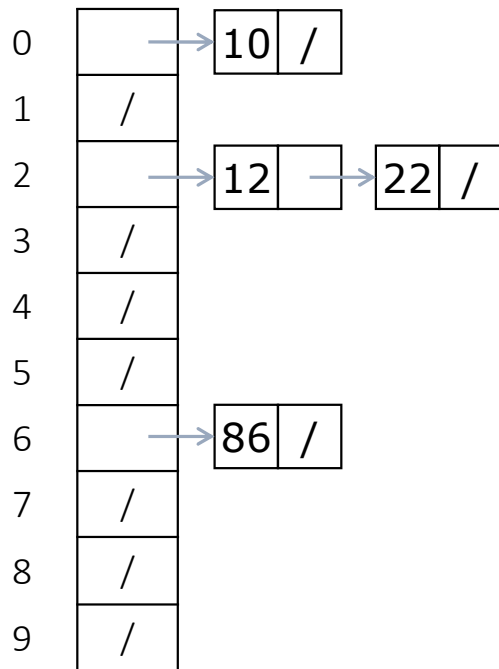
All keys that map to the same table location are kept in a linked list (a.k.a. a "chain" or "bucket")

As easy as it sounds

Example:

insert 10, 22, 86, 12, 42
with $h(x) = x \% 10$

Separate Chaining



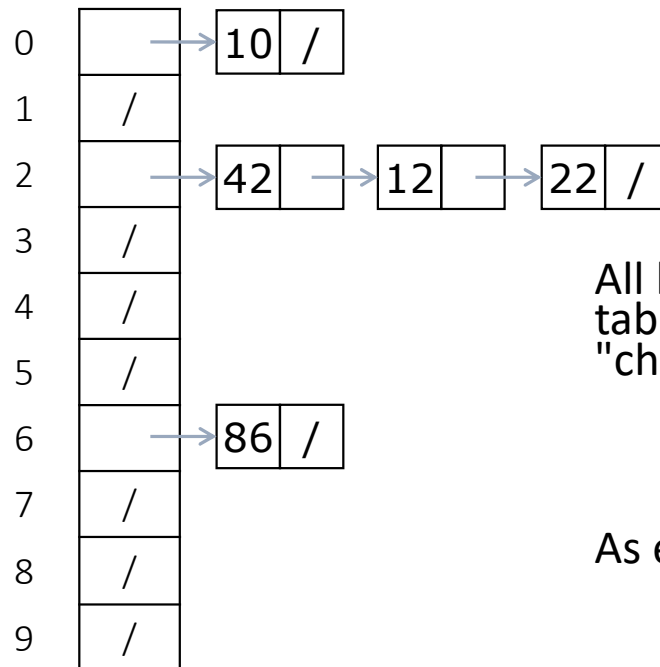
All keys that map to the same table location are kept in a linked list (a.k.a. a "chain" or "bucket")

As easy as it sounds

Example:

insert 10, 22, 86, 12, 42
with $h(x) = x \% 10$

Separate Chaining



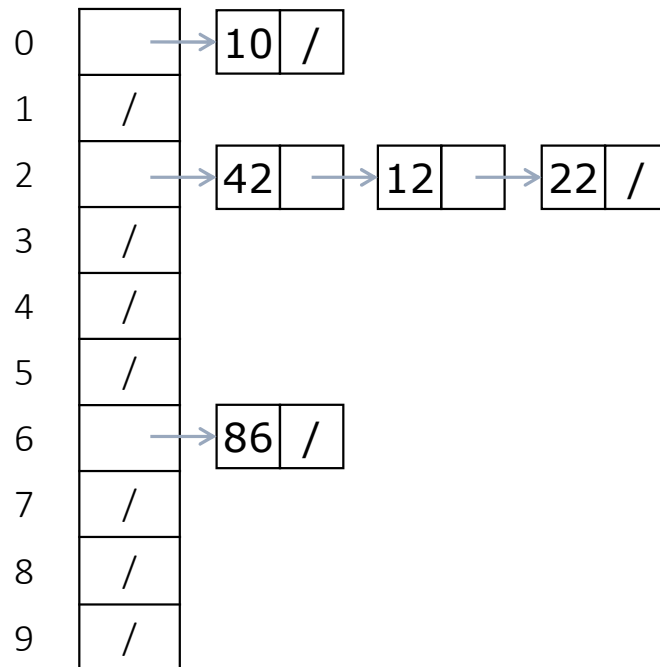
All keys that map to the same table location are kept in a linked list (a.k.a. a "chain" or "bucket")

As easy as it sounds

Example:

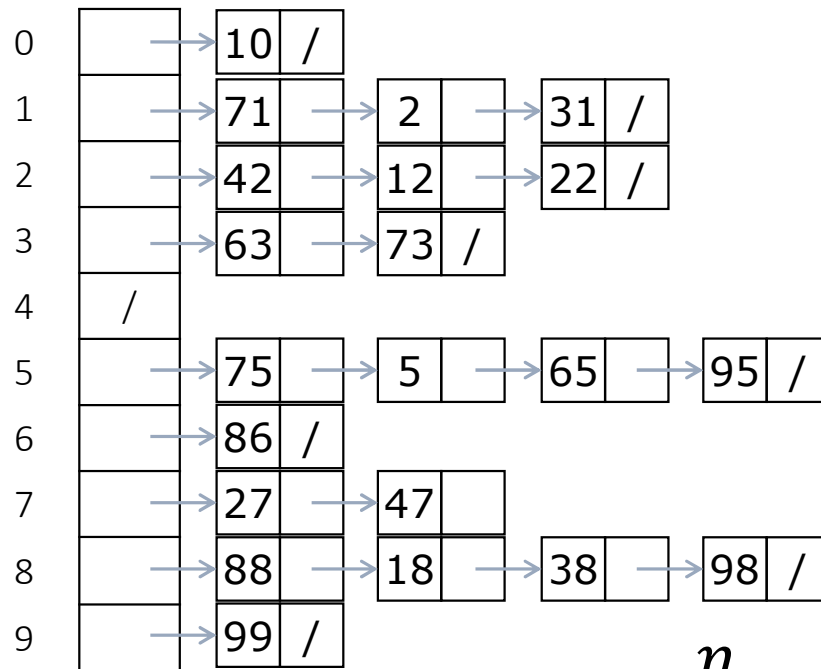
insert 10, 22, 86, 12, 42
with $h(x) = x \% 10$

Load Factor?



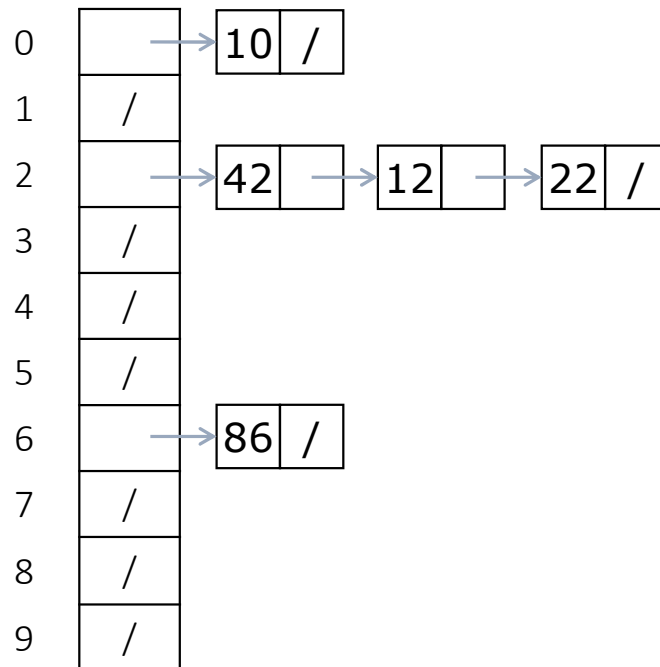
$$\lambda = \frac{n}{TableSize} = \frac{5}{10} = 0.5$$

Load Factor?



$$\lambda = \frac{n}{TableSize} = \frac{21}{10} = 2.1$$

Separate Chaining Deletion



Not too bad and quite easy

- Find in table
- Delete from bucket

Similar run-time as insert

Separate Chaining Animation

<https://liveexample.pearsoncmg.com/dsanimation/SeparateChainingBook.html>

Hashing Separate Chaining Animation by Y. Daniel Liang

Enter the table size and press the enter key to set the hash table size. Entering the load factor threshold factor and press the enter key to set a new load factor threshold. Enter an integer key and click the Search button to search the key in the hash set. Click the Insert button to insert the key into the hash set. Click the Remove button to remove the key from the hash set. Clicking the Remove All button to remove all entries in the hash set. For the best display, use integers between 0 and 99.

Current table size = 11. Number of keys = 4. Current load = 0.36. Load factor threshold = 0.5.

[0]		
[1]	→	1
[2]		
[3]		
[4]	→	48
[5]		
[6]		
[7]		
[8]		
[9]	→	31
[10]	→	21

Enter Initial Table Size: Enter a Load Factor Threshold:

Enter a key:

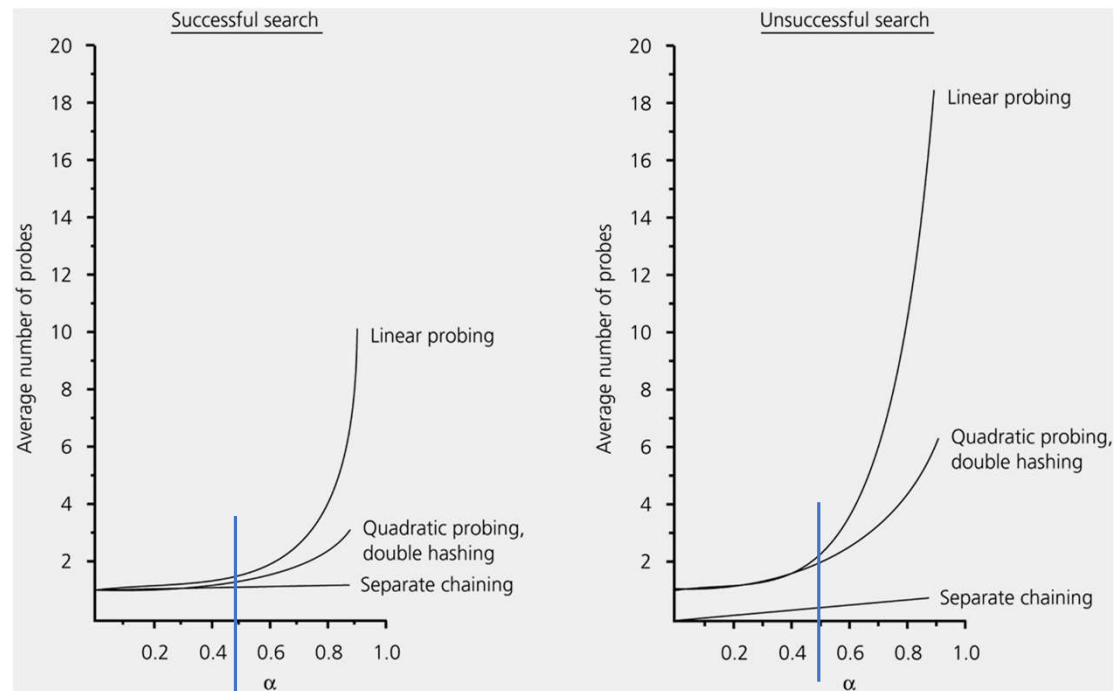
The Efficiency of Hashing

- An analysis of the average-case efficiency of hashing involves the load factor
 - Load factor α
 - Ratio of the current number of items in the table to the maximum size of the array table
 - Measures how full a hash table is
 - Should not exceed $2/3$
 - Hashing efficiency for a particular search also depends on whether the search is successful
 - Unsuccessful searches generally require more time than successful searches

The Efficiency of Hashing

- Linear probing
 - Successful search: $\frac{1}{2}[1 + \frac{1}{1-\alpha}]$
 - Unsuccessful search: $\frac{1}{2}[1 + \frac{1}{(1-\alpha)^2}]$
- Quadratic probing and double hashing
 - Successful search: $\frac{-\log_e(1-\alpha)}{\alpha}$
 - Unsuccessful search: $\frac{1}{1-\alpha}$
- Separate chaining
 - Insertion is $O(1)$
 - Retrievals and deletions
 - Successful search: $1 + \frac{\alpha}{2}$
 - Unsuccessful search: α

The Efficiency of Hashing



The relative efficiency of four collision-resolution methods

What Constitutes a Good Hash Function

- A good hash function should
 - Be easy and fast to compute (using % or bitwise operators)
 - Avoid clusters : Scatter the data evenly throughout the hash table

- Consider:

key $\rightarrow$ nine digits , table-size $\rightarrow$ 40, hashTable[0..39]

$H(\text{key}) \rightarrow (\text{first two digits} \% 40)$



hash code

- Assume keys are unique
- 60 keys will map to hashTable[0..19]
- 40 keys will map to hashTable[20..39]

- Java JCF HashMap provides

```
/** Hash function */
```

```
private int hash(int hashCode) {  
    return supplementalHash(hashCode) & (capacity - 1);  
}
```

```
/** Ensure the hashing is evenly distributed */
```

```
private static int supplementalHash(int h) {  
    h ^= (h >>> 20) ^ (h >>> 12);  
    return h ^ (h >>> 7) ^ (h >>> 4);  
}
```

Exclusive OR Unsigned shift right

For example, both **11100101 & 00000111** and **11001101 & 00000111** yield **00000101**.

But **supplementalHash(11100101) & 00000111** and **supplementalHash(11001101) & 00000111** will be different.

Using a supplemental function reduces this type of collision.